

UNIVERSIDAD EAN

ACTIVIDAD 1

ESTRUCTURAS DE DATOS NO LINEALES

Informe de soluciones: complejidad algorítmica,
árboles binarios y algoritmo de Dijkstra

Estudiante: Sebastian Rene Rodriguez Duarte

Programa: Ingeniería de Sistemas

Materia: Desarrollo de Software

Fecha: 6 de septiembre de 2026

1. Introducción

El presente informe reúne la solución de los ejercicios propuestos en la Actividad 1 “Estructuras de datos no lineales”. El trabajo aborda tres componentes: el análisis de complejidad algorítmica, el estudio y reconstrucción de árboles binarios y la aplicación del algoritmo de Dijkstra para determinar caminos de costo mínimo en un grafo dirigido.

En cada sección se presenta el procedimiento utilizado y la respuesta obtenida, con el propósito de evidenciar la aplicación de los conceptos de estructuras de datos y análisis algorítmico.

2. Complejidad Algorítmica

1. Para cada uno de los tiempos que toma un algoritmo en terminar, demostrar que cada uno es de la complejidad dada:

a) $T_A(n) = 2n^3 - 3n^2 + 1$ es $O(n^3)$

b) $T_A(n) = n^5 + 4^2 - \sqrt{n} + 1$ es $O(n^5)$

c) $T_A(n) = n^2 \log n + 2n^4 + \sqrt{2}n$ es $O(n^4)$

Imagen del ejercicio 1 — Complejidad algorítmica

2.1 Ejercicio 1 — Notación O

Para cada función se identifica el término de mayor crecimiento y se demuestra la cota superior solicitada.

a)
$$T_A(n) = 2n^3 - 3n^2 + 1$$

Solución

Para saber su complejidad debemos mirar qué término crece más rápido cuando aumenta n .

En este caso tenemos los términos $2n^3$, $-3n^2$ y 1 . El término que más crece es $2n^3$, porque n^3 crece más rápido que n^2 y que una constante.

Por esta razón, para determinar la complejidad nos quedamos con el término de mayor grado:

$$2n^3$$

La constante 2 no cambia el orden de crecimiento, por lo que:

$$T_{A(n)} = O(n^3)$$

Respuesta:

$$O(n^3)$$

b)
$$T_A(n) = n^5 + 4^2 - \sqrt{n} + 1 \text{ es } O(n^5)$$

Solución

Primero resolvemos la potencia:

$$4^2 = 16$$

Entonces la función queda:

$$T_{A(n)} = n^5 - \sqrt{\{n\}} + 17$$

Ahora observamos los términos que dependen de n . El término n^5 crece mucho más rápido que $\sqrt{n}$.

Además, el número 17 es una constante, por lo que no afecta el crecimiento de la función cuando n aumenta.

El término dominante es:

$$n^5$$

Por lo tanto:

$$T_{A(n)} = O(n^5)$$

Respuesta:

$$O(n^5)$$

$$\text{c) } T_{A(n)} = n^2 \log n + 2n^4 + \sqrt{2n}$$

En este caso tenemos tres términos principales:

- $n^2 \log n$
- $2n^4$
- $\sqrt{2n}$

Para determinar la complejidad debemos comparar cuál de estos términos crece más rápidamente cuando aumenta n .

El término $\sqrt[3]{(2n)}$ tiene un crecimiento menor que $n^2 \log n$, mientras que n^4 crece más rápidamente que los otros dos términos.

Por lo tanto, el término dominante es:

$$2n^4$$

La constante 2 no cambia el orden de crecimiento, por lo que podemos considerar solamente n^4 .

Así:

$$T_{A(n)} = O(n^4)$$

Respuesta:

$$O(n^4)$$

2. Calcule paso a paso la complejidad del siguiente algoritmo

```
void XXXXXX(int n) {
    int x = 0;

    for (int i = 1; i <= n; i *= 5) {
        int j = 1;
        for (int j = 1; j <= n; j += 2) {
            x = x + j;
        }
        for (int k = n; k >= 1; k /= 2) {
            x = x + 1;
        }
    }
}
```

Imagen del ejercicio 2 — Complejidad algorítmica

2.2 Ejercicio 2 — Complejidad del algoritmo

El algoritmo contiene tres ciclos, de los cuales dos se encuentran dentro de un ciclo externo.

El primero incrementa i multiplicándolo por 5, el segundo incrementa j de 2 en 2 y el tercero divide k entre 2

for ($i = 1$; $i \leq n$; $i *= 5$) $\rightarrow O(\log n)$

for ($j = 1$; $j \leq n$; $j += 2$) $\rightarrow O(n)$

for ($k = n$; $k \geq 1$; $k /= 2$) $\rightarrow O(\log n)$

El ciclo externo se ejecuta $O(\log n)$ veces. En cada iteración se ejecutan un ciclo $O(n)$ y otro $O(\log n)$. Por tanto:

$$O(\log n) \cdot [O(n) + O(\log n)]$$

$$= O(n \log n + \log^2 n)$$

$$= O(n \log n)$$

Como $n \log n$ domina a $\log^2 n$, la complejidad temporal del algoritmo es:

Respuesta:

$$\mathbf{O(n \log n)}$$

3. Suponga que se tienen los siguientes dos algoritmos para resolver el mismo problema, y suponga que la función **buscar** tiene complejidad $O(\log n)$.

```
boolean Ex1(int[] a, int elem) {
    int pos = buscar(a, elem);
    int n = a.length;
    int x = pos;

    for (int i = 0; i < n; ++i) {
        x += 2;
        for (int j = 0; j < n; ++j) {
            if (a[j] > a[pos]) {
                x++;
            }
        }
    }
    return x > elem;
}
```

```
boolean Ex2(int[] a, int elem) {
    int n = a.length;
    int x = 0;

    for (int i = 0; i < n; ++i) {
        int pos = buscar(a, elem);
        x += pos + 2;
        for (int j = 0; j < n; ++j) {
            if (a[j] > a[pos]) {
                x++;
            }
        }
    }
    return x > elem;
}
```

Indique con cuál de los dos algoritmos se queda para resolver el problema. Justifique muy bien su respuesta.

Imagen del ejercicio 3 — Complejidad algorítmica

2.3 Ejercicio 3 — Selección del algoritmo

Algoritmo 1

La función `buscar()` tiene una complejidad $O(\log n)$ y se ejecuta una sola vez. Luego encontramos dos ciclos anidados que se ejecutan n veces cada uno:

$$O(n) \cdot O(n) = O(n^2)$$

Por lo tanto:

$$O(\log n) + O(n^2) = O(n^2)$$

Algoritmo 2

En este caso, la función `buscar()` está dentro del ciclo principal, por lo que se ejecuta n veces:

$$O(n) \cdot O(\log n) = O(n \log n)$$

Además, existen dos ciclos anidados que generan:

$$O(n^2)$$

Por lo tanto:

$$O(n \log n) + O(n^2) = O(n^2)$$

Comparación de los dos algoritmos

Tenemos:

$$\text{Algoritmo 1} = O(n^2)$$

$$\text{Algoritmo 2} = O(n^2)$$

Los dos algoritmos tienen la misma complejidad asintótica, es decir, ambos son $O(n^2)$.

Sin embargo, hay una diferencia importante. En el Algoritmo 1, la función `buscar()` se ejecuta una sola vez, mientras que en el Algoritmo 2 se ejecuta n veces.

Por esta razón, el Algoritmo 2 realiza operaciones adicionales de complejidad $O(n \log n)$ que no son necesarias en el Algoritmo 1.

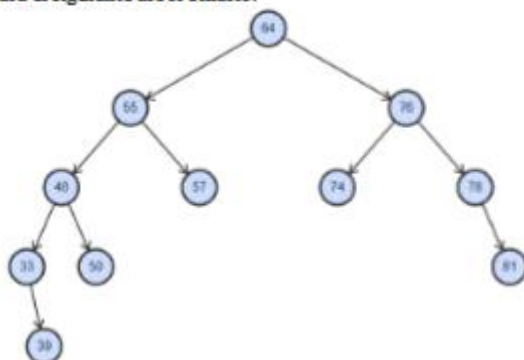
Respuesta

Se selecciona el Algoritmo 1

Justificación: Se selecciona el Algoritmo 1 porque, aunque ambos algoritmos tienen una complejidad de $O(n^2)$, el primero realiza la búsqueda una sola vez. En cambio, el segundo ejecuta la función buscar() en cada iteración del ciclo principal, realizando un mayor número de operaciones. Por lo tanto, el Algoritmo 1 resulta más eficiente en la práctica.

3. ÁRBOLES BINARIOS

1. Para el siguiente árbol binario:



Presente:

- a. Peso:
- b. Altura:
- c. Hojas:
- d. Una rama:
- e. Recorrido en inorden
- f. Recorrido en preorden
- g. Recorrido en postorden

Imagen del ejercicio 1 — Árboles binarios

3.1 Ejercicio 1

Para el árbol cuya raíz es 64:

- Peso: 11 nodos.
- Altura: 4 aristas, equivalente a 5 niveles si se cuenta la raíz como nivel 1.
- Hojas: 39, 50, 57, 74 y 81 (5 hojas).

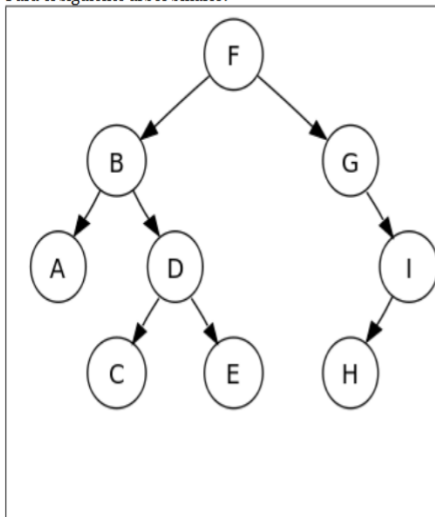
- Una rama completa: $64 \rightarrow 55 \rightarrow 48 \rightarrow 33 \rightarrow 39$.

Recorrido en inorden: $33 - 39 - 48 - 50 - 55 - 57 - 64 - 74 - 76 - 78 - 81$

Recorrido en preorden: $64 - 55 - 48 - 33 - 39 - 50 - 57 - 76 - 74 - 78 - 81$

Recorrido en postorden: $39 - 33 - 50 - 48 - 57 - 55 - 74 - 81 - 78 - 76 - 64$

2. Para el siguiente árbol binario:



Indique:

- Altura:
- Número de niveles:
- Ancestro común de la E y la A
- Peso del árbol izquierdo de la F
- Recorrido en inorden
- Recorrido en preorden
- Recorrido en postorden
- Recorrido por niveles
- Hojas

Imagen del ejercicio 2 — Árboles binarios

3.2 Ejercicio 2

- Altura: 3 aristas, equivalente a 4 niveles.
- Número de niveles: 4.
- Ancestro común de E y A: F.
- Peso del árbol izquierdo de F: 5 nodos.
- Hojas: A, C, E y H (4 hojas).

Inorden: $A - B - C - D - E - F - G - H - I$

Preorden: $F - B - A - D - C - E - G - I - H$

Postorden: $A - C - E - D - B - H - I - G - F$

separamos los elementos que pertenecen al subárbol izquierdo y al subárbol derecho. Este procedimiento se repite de manera recursiva hasta ubicar todos los nodos.

4. Reconstruya el árbol binario que posee los siguientes recorridos:

Postorden: A – C – E – D – B – H – I – G – F

Inorden: A – B – C – D – E – F – G – H – I

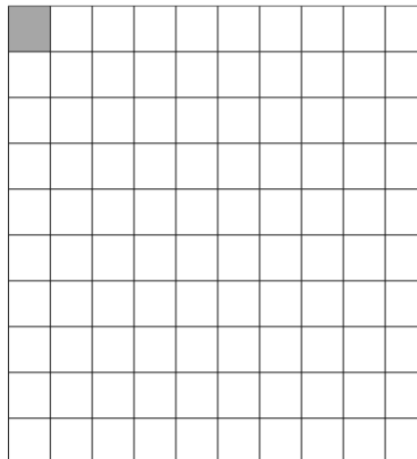


Imagen del ejercicio 4 — Árboles binarios

3.4 Ejercicio 4 — Reconstrucción

Postorden: A – C – E – D – B – H – I – G – F

Inorden: A – B – C – D – E – F – G – H – I

```

      F
    /  \
   B    G
  /\   \
 A  D   I
 /\   /
C  E H

```

En postorden, el último elemento corresponde a la raíz del árbol, por lo que F es la raíz. Luego, al ubicar F en el recorrido inorden, se separan los elementos que pertenecen al subárbol izquierdo

y al subárbol derecho. Este procedimiento se repite de forma recursiva hasta ubicar todos los nodos.

5. Reconstruya el árbol binario que posee los siguientes recorridos:

Preorden: 59 – 37 – 28 – 16 – 43 – 48 – 74 – 80 – 79

Inorden: 16 – 28 – 37 – 43 – 48 – 59 – 74 – 79 – 80

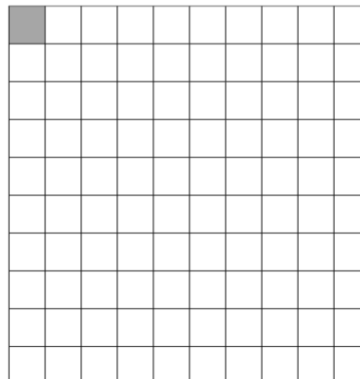


Imagen del ejercicio 5 — Árboles binarios

3.5 Ejercicio 5 — Reconstrucción

Preorden: 59 – 37 – 28 – 16 – 43 – 48 – 74 – 80 – 79

Inorden: 16 – 28 – 37 – 43 – 48 – 59 – 74 – 79 – 80

```

      59
     / \
    37  74
   / \  \
  28 43 80
 /   \ /
16   48 79

```

La raíz es 59 porque es el primer elemento del recorrido en preorden. Luego, al ubicar 59 en el recorrido inorden, se separan los elementos que pertenecen al subárbol izquierdo y al subárbol derecho. Este procedimiento se repite de forma recursiva hasta ubicar todos los nodos.

4. Algoritmo de Dijkstra

Se aplica el algoritmo de Dijkstra al grafo dirigido suministrado en la actividad, tomando A como vértice de origen. En cada iteración se selecciona el vértice no seleccionado con menor costo provisional y se relajan sus aristas salientes.

Para el siguiente grafo:

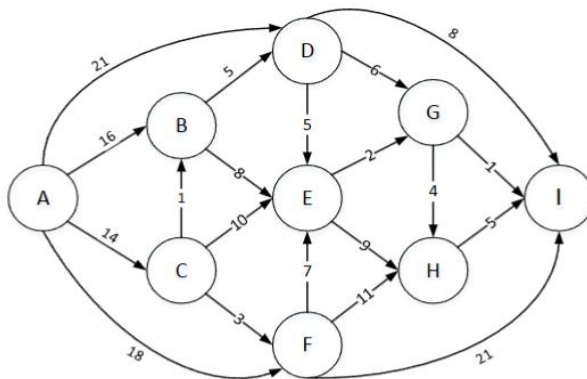


Imagen del ejercicio — Algoritmo de Dijkstra

4.1 Desarrollo de las iteraciones

Iteración 1

Vértice seleccionado: A Seleccionados: A

	A	B	C	D	E	F	G	H	I
Costo	0	16	14	21	∞	18	∞	∞	∞
Anterior	—	A	A	A	—	A	—	—	—

Actualizaciones: B=16 (anterior A); C=14 (anterior A); D=21 (anterior A); F=18 (anterior A)

Iteración 2

Vértice seleccionado: C Seleccionados: A, C

	A	B	C	D	E	F	G	H	I
--	---	---	---	---	---	---	---	---	---

Costo	0	15	14	21	24	17	∞	∞	∞
Anterior	—	C	A	A	C	C	—	—	—

Actualizaciones: B=15 (anterior C); E=24 (anterior C); F=17 (anterior C)

Iteración 3

Vértice seleccionado: B **Seleccionados:** A, C, B

	A	B	C	D	E	F	G	H	I
Costo	0	15	14	20	23	17	∞	∞	∞
Anterior	—	C	A	B	B	C	—	—	—

Actualizaciones: D=20 (anterior B); E=23 (anterior B)

Iteración 4

Vértice seleccionado: F **Seleccionados:** A, C, B, F

	A	B	C	D	E	F	G	H	I
Costo	0	15	14	20	23	17	∞	28	38
Anterior	—	C	A	B	B	C	—	F	—

Actualizaciones: H = 28 (anterior F); I = 38 (anterior F)

Iteración 5

Vértice seleccionado: D **Seleccionados:** A, C, B, F, D

	A	B	C	D	E	F	G	H	I
Costo	0	15	14	20	23	17	26	28	28
Anterior	—	C	A	B	B	C	D	F	D

Actualizaciones: G=26 (anterior D); I=28 (anterior D)

Iteración 6**Vértice seleccionado:** E **Seleccionados:** A, C, B, F, D, E

	A	B	C	D	E	F	G	H	I
Costo	0	15	14	20	23	17	25	28	28
Anterior	—	C	A	B	B	C	E	F	D

Actualizaciones: G=25 (anterior E)

Iteración 7**Vértice seleccionado:** G **Seleccionados:** A, C, B, F, D, E, G

	A	B	C	D	E	F	G	H	I
Costo	0	15	14	20	23	17	25	28	26
Anterior	—	C	A	B	B	C	E	F	G

Actualización: I = 26 (anterior G)

Iteración 8**Vértice seleccionado:** I **Seleccionados:** A, C, B, F, D, E, G, I

	A	B	C	D	E	F	G	H	I
Costo	0	15	14	20	23	17	25	28	26
Anterior	—	C	A	B	B	C	E	F	G

No se producen nuevas actualizaciones.

Iteración 9**Vértice seleccionado:** H **Seleccionados:** A, C, B, F, D, E, G, H, I

	A	B	C	D	E	F	G	H	I
--	---	---	---	---	---	---	---	---	---

Costo	0	15	14	20	23	17	25	28	26
Anterior	—	C	A	B	B	C	E	F	G

No se producen nuevas actualizaciones.

4.2 Resultado final

Vértice	Costo mínimo desde A	Anterior
A	0	—
B	15	C
C	14	A
D	20	B
E	23	B
F	17	C
G	25	E
H	28	F
I	26	G

Para obtener el camino $A \rightarrow I$ se reconstruye la ruta desde I utilizando el campo Anterior: $I \leftarrow G$

$\leftarrow E \leftarrow B \leftarrow C \leftarrow A$. Al invertir la secuencia se obtiene:

$A \rightarrow C \rightarrow B \rightarrow E \rightarrow G \rightarrow I$

Costo mínimo total = 26

Cálculo: $A \rightarrow C = 14$, $C \rightarrow B = 1$, $B \rightarrow E = 8$, $E \rightarrow G = 2$ y $G \rightarrow I = 1$. Por tanto, $14+1+8+2+1=26$

5. Conclusiones

- El análisis de complejidad permite identificar el comportamiento de los algoritmos a medida que aumenta el tamaño de la entrada y facilita seleccionar soluciones con mejor escalabilidad.
- Los recorridos en inorden, preorden, postorden y por niveles permiten procesar sistemáticamente los nodos de un árbol binario y, junto con los recorridos conocidos, reconstruir su estructura.
- El algoritmo de Dijkstra permite determinar caminos de costo mínimo desde un vértice origen cuando los pesos de las aristas son no negativos. En el grafo analizado, el camino mínimo entre A e I es $A \rightarrow C \rightarrow B \rightarrow E \rightarrow G \rightarrow I$, con costo 26.

6. Referencias bibliográficas

- Villalobos, J. (1996). Estructura de datos. Material bibliográfico indicado en la actividad.
- Weiss, M. A. (2012). Data Structures and Algorithm Analysis. Material bibliográfico indicado en la actividad.
- Centripio (2019). Material audiovisual sobre complejidad algorítmica. Recurso indicado en la actividad.
- Tentor (2015e). Material audiovisual sobre árboles binarios, reconstrucción y recorridos. Recurso indicado en la actividad.
- Universidad Politécnica de Valencia (UPV) (2013). Material audiovisual sobre el algoritmo de Dijkstra. Recurso indicado en la actividad.